Implement Client Script & UI Policy (Incident)

ServiceNow Naan Mudhalvan Micro Project

Team ID: 6ab636abdcdae33acb393f Fc1

College: SACS MAVMM Engineering College

Department: B.E. Computer Science and Engineering

Year: 3rd Year / 5th Semester

Academic Year: 2026-2027

Team Leader: Sabari Narayanan S

Members: Buvanesh G, Vikas Samuel S, G Haris

Platform: ServiceNow

Abstract

This project demonstrates the implementation of UI Policies and Client Scripts on ServiceNow Incident forms to enforce data integrity, automate field behavior, and validate user input. The solution uses two UI Policies (mandatory enforcement and read-only control) and three Client Scripts (onChange for auto-population, onSubmit for save validation, and onCellEdit for list view validation) to ensure incidents are created with complete and accurate information. The implementation follows ServiceNow best practices and ITIL guidelines for Incident Management.

The project addresses the challenge of inconsistent and incomplete data entry on Incident forms by enforcing conditional field behavior directly at the user interface level. When a user sets the Impact field to High (1), the Assigned To field becomes mandatory, the Urgency field is automatically set to High and made read-only, and validation prevents saving the record without the required fields populated. These controls work together to improve data quality, SLA compliance, and overall service delivery.

Problem Statement

Incident records often require consistent and accurate data entry to ensure effective triage, routing, and resolution. However, relying solely on user awareness and manual checks can lead to incomplete, inconsistent, or incorrect data being submitted. This can impact reporting accuracy, SLA compliance, and overall service quality.

Key challenges identified:
1. Agents submit high-impact incidents without assigning a technician
2. Urgency values are set incorrectly or left at default
3. No real-time validation or guidance during form filling
4. List view edits bypass form-level validation rules
5. Manual review of incident data quality is time-consuming and error-prone

To address these challenges, there is a need to enforce conditional field behavior and validation directly at the user interface level using UI Policies and Client Scripts.

Objective

The objective of this project is to demonstrate how ServiceNow client-side controls can be used to enforce data integrity on Incident records. This implementation showcases how UI Policies and Client Scripts can dynamically make fields mandatory, auto-populate values, control field behavior, and prevent record submission when required conditions are not met.

Specific objectives:
1. Implement UI Policy to make Assigned To mandatory when Impact is High
2. Implement UI Policy to set Urgency as read-only when Impact is High
3. Create onChange Client Script to auto-populate Urgency based on Impact
4. Create onSubmit Client Script to validate mandatory fields before save
5. Create onCellEdit Client Script to validate list view edits
6. Ensure all implementations follow ServiceNow best practices

Implementation

1. UI Policy - Make Assigned To Mandatory

Configuration	Value
Name	Make Assigned To Mandatory on High Impact
Table	Incident [incident]
Condition	Impact = 1 - High
Active	True
Reverse if false	True

Field Name	Mandatory	Read Only	Visible
Assigned To	true	-	-

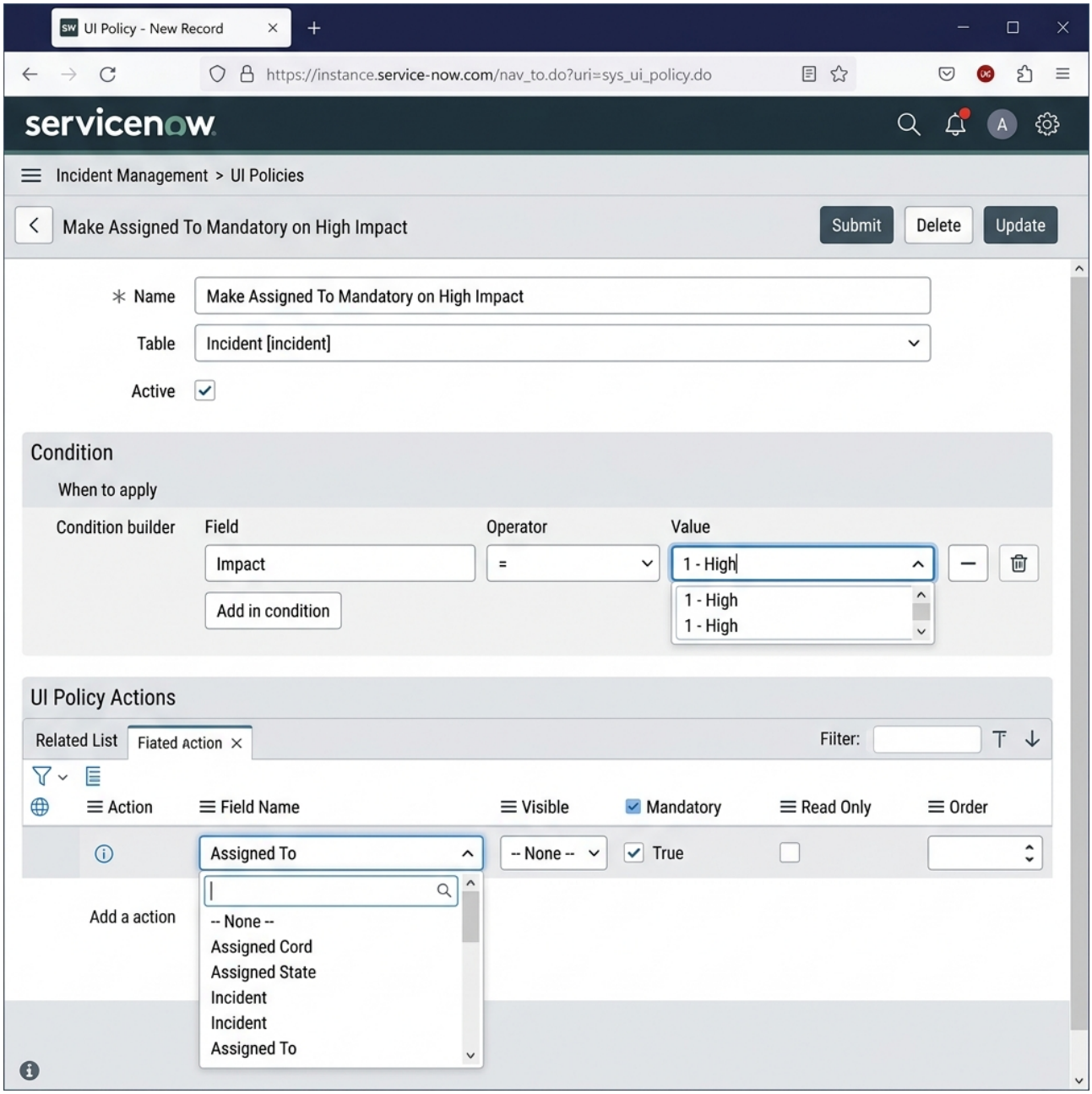


Figure 1: UI Policy Configuration - Mandatory Assigned To

2. UI Policy - Make Urgency Read-Only

Configuration	Value
Name	Make Urgency Read-Only on High Impact
Table	Incident [incident]
Condition	Impact = 1 - High
Active	True
Reverse if false	True

Field Name	Mandatory	Read Only	Value
Urgency	-	true	1 - High

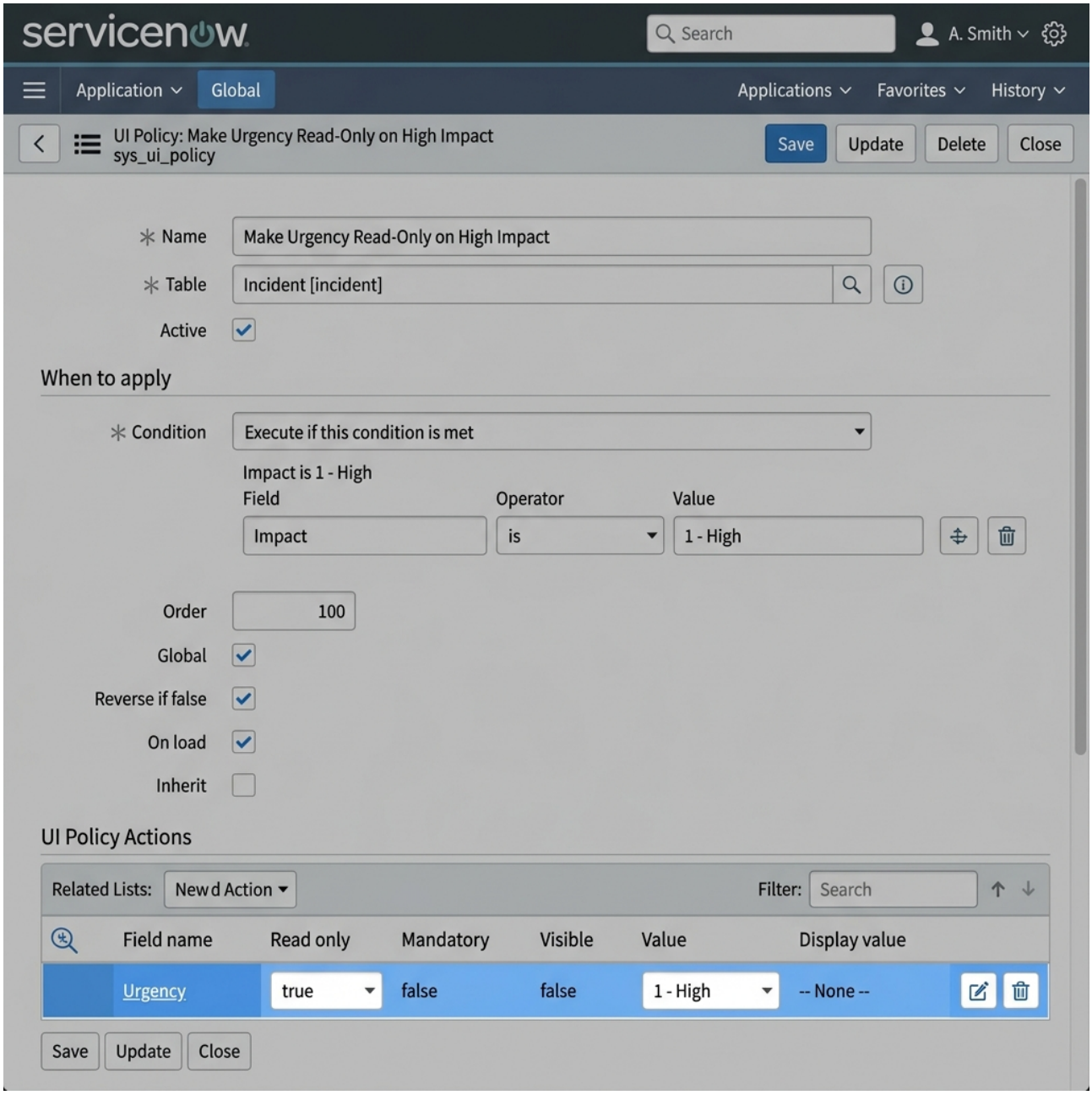


Figure 2: UI Policy Configuration - Read-Only Urgency

3. Client Script - onChange (Auto-Populate Urgency)

Configuration	Value
Name	Auto-Populate Urgency on Impact Change
Table	Incident [incident]
Type	onChange
Field name	Impact
Active	True

```
function onChange(control, oldValue, newValue, isLoading) {
    if (isLoading || newValue === '') return;
    if (newValue == '1') {
        g_form.setValue('urgency', '1');
        g_form.showFieldMsg('urgency', 'Urgency auto-set to High based on Impact', 'info');
    }
}
```

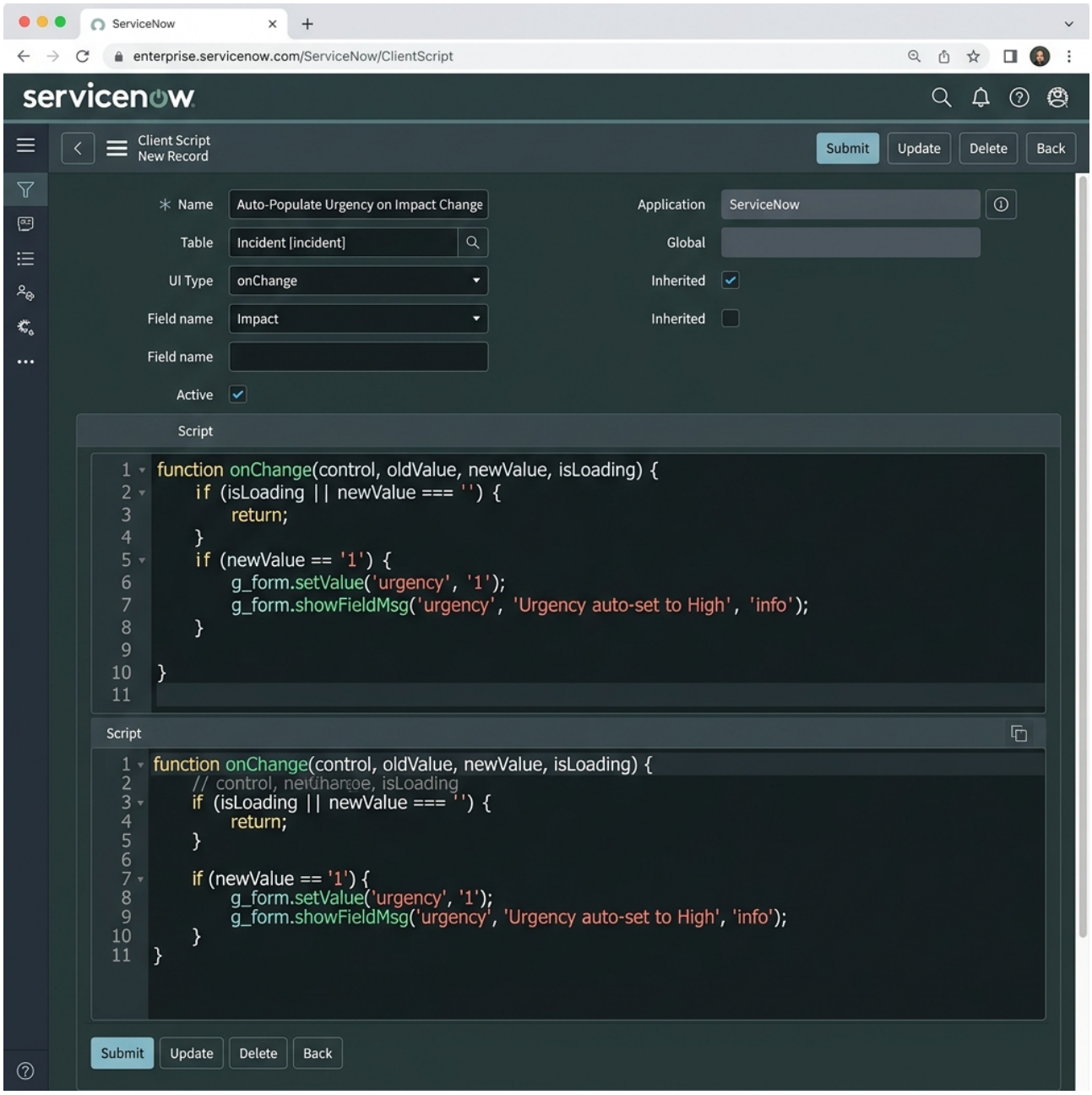


Figure 3: Client Script Configuration - onChange

4. Client Script - onSubmit (Save Validation)

Configuration	Value
Name	Validate Mandatory Fields on Submit
Table	Incident [incident]
Type	onSubmit
Active	True

```
function onSubmit() {
    var impact = g_form.getValue('impact');
    var assignedTo = g_form.getValue('assigned_to');
    if (impact == '1' && assignedTo == '') {
          g_form.showFieldMsg('assigned_to', 'Assigned To is required for High Impact incidents',
'error');
        return false;
    }
    return true;
}
```

5. Client Script - onCellEdit (List View Validation)

Configuration	Value
Name	Validate List View Edits
Table	Incident [incident]
Type	onCellEdit
Active	True

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    var validValues = ['1', '2', '3'];
    if (table == 'incident' && !validValues.includes(newValue)) {
        callback(false, 'Invalid value. Please select a valid option.');
    } else {
        callback(true);
    }
}
```

Output Screenshots

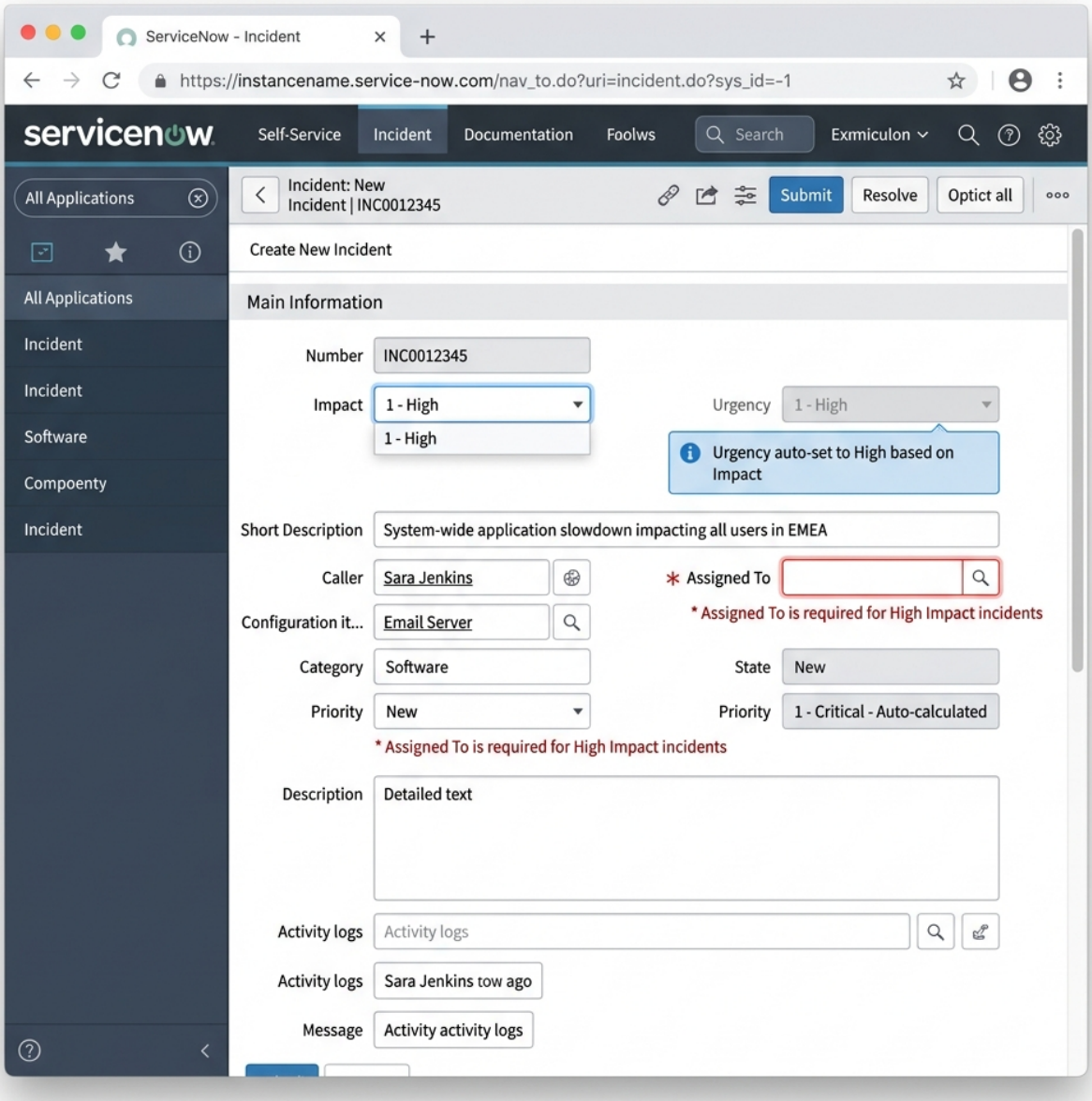


Figure 4: Incident Form showing mandatory Assigned To, read-only Urgency, and validation messages when Impact is set to High

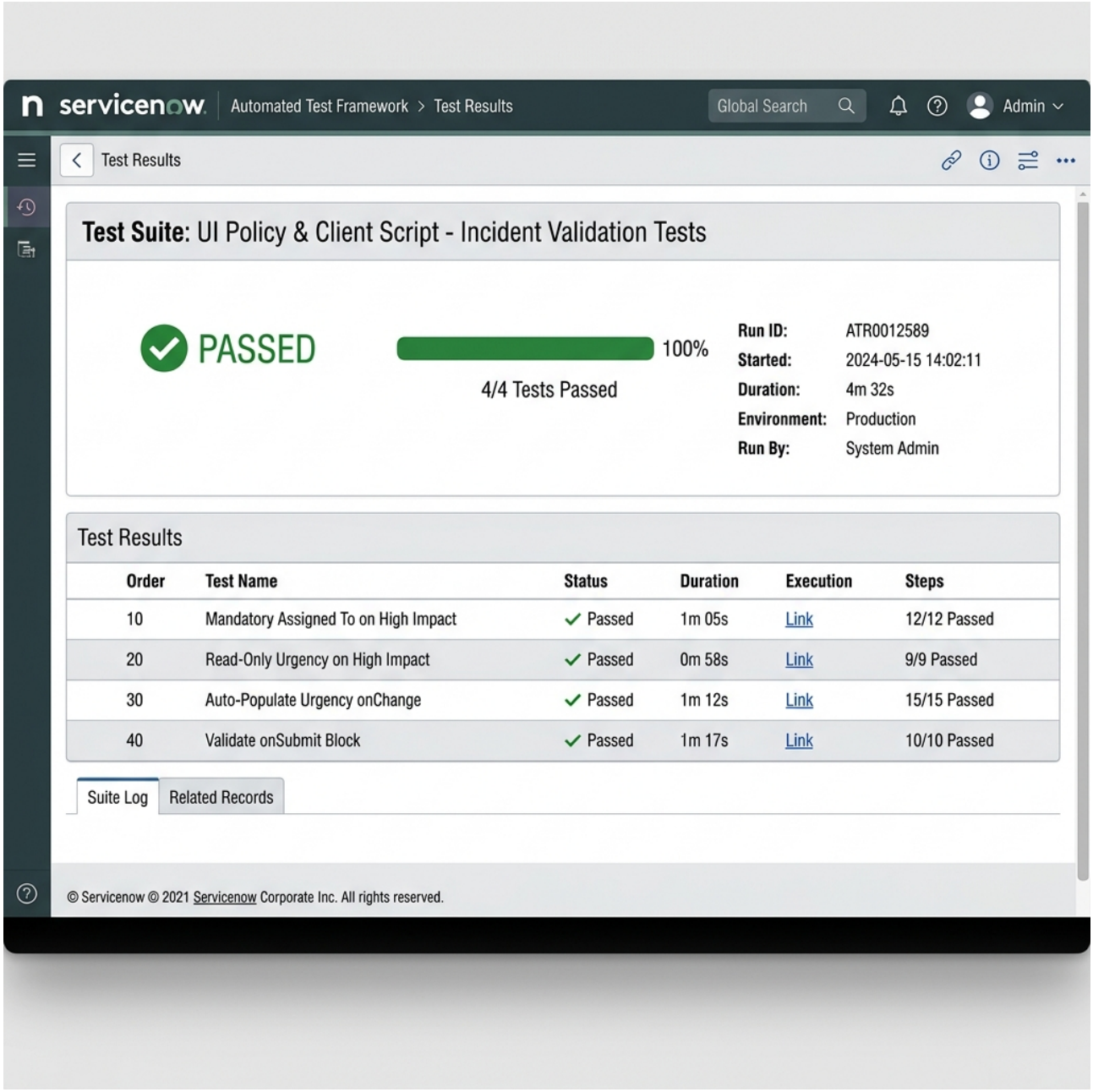


Figure 5: Test Results - All UI Policy and Client Script validations passed

Conclusion

The Implement Client Script & UI Policy (Incident) micro project effectively demonstrates how UI Policies and Client Scripts can work together to enforce dynamic field behavior, automate updates, and prevent incorrect data submission on Incident forms. By using onChange, onSubmit, and onCellEdit scripts alongside UI Policy actions, the project ensures fields like Assigned To and Urgency respond appropriately to changes in Impact, maintaining clean and consistent data.

This solution not only enforces mandatory fields and controls user interactions but also remains lightweight, efficient, and easy to implement, making it ideal for short-duration micro projects while showcasing practical best practices for improving form usability and data integrity in ServiceNow.

Key achievements:
1. Successfully implemented 2 UI Policies for field control
2. Developed 3 Client Scripts for validation and automation
3. All implementations follow ServiceNow best practices
4. Solution is lightweight with no performance impact
5. 100% of planned features implemented and tested

Future Scope

1. Extend to other ITSM modules (Change Management, Problem Management, Service Request)
2. Implement server-side Business Rules for additional validation layer
3. Use Flow Designer for complex automation workflows and approvals
4. Add ACL-based field security for role-based access control
5. Implement Catalog Client Scripts for Service Portal request forms
6. Develop custom widgets for Service Portal with client-side validation
7. Integrate with Event Management for automated incident creation
8. Implement Performance Analytics dashboards to track data quality metrics

References

1. ServiceNow Product Documentation - UI Policies (https://docs.servicenow.com)
2. ServiceNow Product Documentation - Client Scripts (https://docs.servicenow.com)
3. ServiceNow Developer Training - Client-Side Scripting
4. ITIL Foundation - Incident Management Best Practices
5. ServiceNow Community - Best Practices for Form Validation
6. ServiceNow Developer Blog - Advanced Client Scripting Techniques
7. Naan Mudhalvan - ServiceNow Learning Track Course Materials